

Thực hành 4: Tạo giao diện ứng dụng web với Angular

Table of Contents

1. Mục đích	2
2. Mô tả ứng dụng web cần xây dựng:.....	2
2.1 Trang web liệt kê các công việc:	2
2.2 Trang web hiển thị chi tiết của một công việc:.....	2
2.3 Trang Thêm công việc:.....	3
2.4 Trang Page Not Found:	3
3. Chuẩn bị môi trường phát triển ứng dụng Angular	4
4. Tạo bộ khung cho project.....	4
4.1 Khởi tạo project:	4
4.2 Tạo các component:	5
4.3 Thiết lập chuyển hướng (routing) cho các trang web:	6
5. Xử lý các component	9
5.1 Tạo nguồn dữ liệu cho website:	9
5.2 Xử lý trang hiển thị danh sách:	10
5.2.1 Hiển thị danh sách công việc	10
5.2.2 Xử lý các sự kiện	12
5.2.3 Kết quả:.....	13
5.3 Xử lý trang thêm mới công việc:.....	14
5.3.1 Tạo form nhập dữ liệu:	14
5.3.2 Xử lý sự kiện:	16
5.3.3 Kết quả:.....	16
5.4 Xử lý trang hiển thị chi tiết công việc	18
5.4.1 Hiển thị chi tiết công việc:	19
5.4.2 Xử lý sự kiện:	20
5.4.3 Tổng kết:	21

1. Mục đích:

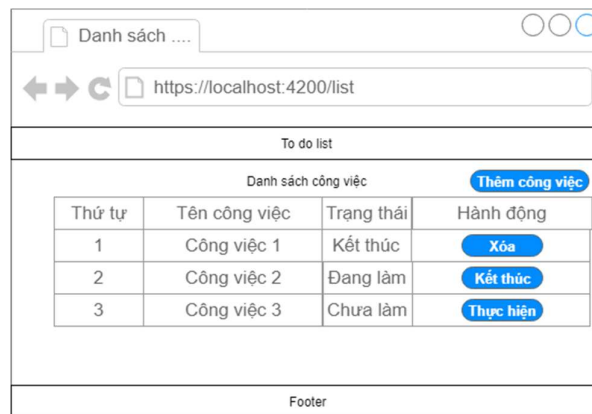
Làm quen với Angular và môi trường phát triển ứng dụng với Angular. Tạo lập giao diện của một ứng dụng web đơn giản.

2. Mô tả ứng dụng web cần xây dựng:

Ứng dụng web là một trang quản lý công việc đơn giản, có các chức năng hiển thị danh sách công việc cần làm, thêm công việc mới, sửa xóa công việc, đánh dấu công việc đã hoàn thành. Giao diện của ứng dụng gồm các trang sau (tất cả các trang đều có chung phần header và footer):

2.1 Trang web liệt kê các công việc:

Hiển thị các công việc dưới dạng danh sách, trong đó mỗi công việc cần thể hiện các thông tin: tiêu đề, trạng thái công việc.



To do list			
Danh sách công việc			Thêm công việc
Thứ tự	Tên công việc	Trạng thái	Hành động
1	Công việc 1	Kết thúc	Xóa
2	Công việc 2	Đang làm	Kết thúc
3	Công việc 3	Chưa làm	Thực hiện
Footer			

Bên cạnh mỗi công việc là một nút hành động để cập nhật thông tin của công việc. Khi nhấn vào tiêu đề của công việc thì mở ra trang hiển thị chi tiết công việc. Trên trang này còn có nút Thêm công việc, khi nhấn vào sẽ mở ra trang Thêm công việc.

2.2 Trang web hiển thị chi tiết của một công việc:

Trình bày các thông tin chi tiết về một công việc: tiêu đề, nội dung, thời hạn, trạng thái. Bên dưới phần hiển thị là các nút chức năng gồm Xóa, Kết thúc (hiển thị khi trạng thái công việc là “Đang làm”), Thực hiện (hiển thị khi trạng thái công việc là “Chưa làm”). Ngoài ra còn có thêm nút Back để quay trở lại trang danh sách.

Danh sách

https://localhost:4200/detail/1

To do list

Back

Chi tiết công việc

Công việc 1

Nội dung

Lorem ipsum dolor sit amet, consectetur adipisicing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat.

Đang thực hiện

Xóa

Kết thúc

Footer

2.3 Trang Thêm công việc:

Trang này là một form gồm các ô nhập liệu để nhập các thông tin về một công việc. Có 2 nút trên trang này: nút Lưu để lưu công việc và quay trở về trang danh sách, nút Hủy để trở về trang danh sách.

Danh sách

https://localhost:4200/new

To do list

Thêm công việc mới

Tiêu đề

Nội dung

Hủy

Lưu

Footer

2.4 Trang Page Not Found:

Trang mặc định cho các URL không hợp lệ

Page not

https://localhost:4200/pagenotfound

To do list

Page Not Found!

Footer

3. Chuẩn bị môi trường phát triển ứng dụng Angular

Theo hướng dẫn trên trang elearning của môn học

4. Tạo bộ khung cho project

Ở bước này ta sẽ tạo project cho ứng dụng Angular với các component cần thiết.

4.1 Khởi tạo project:

Chọn một thư mục để đặt project, chú ý lựa chọn sao cho đường dẫn đến thư mục này ngắn gọn và không chứa ký tự đặc biệt. VD: D:\SV\SV\DemoLTUDM

Mở chương trình Command Prompt (tìm chương trình này trong danh sách phần mềm cài đặt trên máy hoặc nhấn Windows+R để mở cửa sổ Run, sau đó gõ cmd vào ô Open và nhấn OK), đổi thư mục làm việc về thư mục chứa project bằng cách sử dụng lệnh `cd <thư mục>` . Ví dụ:

```
C:\WINDOWS\system32\cmd.exe
Microsoft Windows [Version 10.0.17134.345]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Users\Duc Quang>cd D:\SV\SV\DemoLTUDM

C:\Users\Duc Quang>D:

D:\SV\SV\DemoLTUDM>
```

Tạo project Angular mới bằng lệnh **ng new <tên project>**

Ví dụ: **ng new thuchanh1**

Chọn N cho tùy chọn routing và CSS cho tùy chọn style

```
D:\SV\SV\DemoLTUDM>ng new thuchanh1
? Would you like to add Angular routing? No
? Which stylesheet format would you like to use? CSS
```

Đợi project khởi tạo xong rồi gõ tiếp lệnh sau để chạy project:

- chuyển thư mục làm việc đến thư mục của project bằng lệnh cd.

VD: cd thuchanh1

- Chạy project bằng lệnh **ng serve --open**

```
D:\SV\SV\DemoLTUDM>cd thuchanh1
D:\SV\SV\DemoLTUDM\thuchanh1>ng serve --open
** Angular Live Development Server is listening on localhost:4200, open your browser on http://localhost:4200/ **
```

Sau khi website mở thành công trên trình duyệt tại địa chỉ localhost:4200, ta giữ nguyên màn hình Command Prompt này và mở một màn hình Command Prompt khác để thao tác các bước tiếp theo.

4.2 Tạo các component:

Mỗi component là một khối hiển thị trên trang web, để sử dụng cơ chế chuyển hướng của Angular, ta cần tạo mỗi component cho từng trang: danh sách, chi tiết, thêm mới và trang báo lỗi.

Lần lượt tạo các component bằng lệnh sau: **ng generate component <tên component>**

Chú ý tạo mới các component trên một Command Prompt khác, cũng chuyển hướng về thư mục project như Command Prompt ban đầu.

VD: tạo các component task-list, task-detail, task-new, page-not-found

```
Microsoft Windows [Version 10.0.17134.345]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Users\Duc Quang>cd D:\SV\SV\DemoLTUDM\thuchanh1

C:\Users\Duc Quang>D:

D:\SV\SV\DemoLTUDM\thuchanh1>ng generate component task-list
CREATE src/app/task-list/task-list.component.html (28 bytes)
CREATE src/app/task-list/task-list.component.spec.ts (643 bytes)
CREATE src/app/task-list/task-list.component.ts (280 bytes)
CREATE src/app/task-list/task-list.component.css (0 bytes)
UPDATE src/app/app.module.ts (406 bytes)

D:\SV\SV\DemoLTUDM\thuchanh1>ng generate component task-detail
CREATE src/app/task-detail/task-detail.component.html (30 bytes)
CREATE src/app/task-detail/task-detail.component.spec.ts (657 bytes)
CREATE src/app/task-detail/task-detail.component.ts (288 bytes)
CREATE src/app/task-detail/task-detail.component.css (0 bytes)
UPDATE src/app/app.module.ts (506 bytes)

D:\SV\SV\DemoLTUDM\thuchanh1>ng generate component task-new
CREATE src/app/task-new/task-new.component.html (27 bytes)
CREATE src/app/task-new/task-new.component.spec.ts (636 bytes)
CREATE src/app/task-new/task-new.component.ts (276 bytes)
CREATE src/app/task-new/task-new.component.css (0 bytes)
UPDATE src/app/app.module.ts (594 bytes)

D:\SV\SV\DemoLTUDM\thuchanh1>ng generate component page-not-found
CREATE src/app/page-not-found/page-not-found.component.html (33 bytes)
CREATE src/app/page-not-found/page-not-found.component.spec.ts (672 bytes)
CREATE src/app/page-not-found/page-not-found.component.ts (299 bytes)
CREATE src/app/page-not-found/page-not-found.component.css (0 bytes)
UPDATE src/app/app.module.ts (704 bytes)
```

Kết quả ta sẽ có các component được sinh ra bên trong thư mục /src/app.

4.3 Thiết lập chuyển hướng (routing) cho các trang web:

Ở bước trước ta đã tạo ra các component tương ứng với mỗi trang web, bước này ta sẽ cấu hình để hiển thị mỗi trang web khi gõ địa chỉ trang web vào thanh địa chỉ. Ứng dụng web mặc định sẽ có đường dẫn gốc là <http://localhost:4200> với trang web được hiển thị mặc định là nội dung của file /src/index.html. Mở file này ra ta sẽ thấy phần <body> của trang này là component gốc <app-root> của ứng dụng.

```
<!doctype html>

<html lang="en">

<head>

  <meta charset="utf-8">

  <title>Thuchanh1</title>
```

```
<base href="/">

<meta name="viewport" content="width=device-width,
initial-scale=1">

<link rel="icon" type="image/x-icon" href="favicon.ico">
</head>
<body>

  <app-root></app-root>
</body>
</html>
```

Theo cơ chế routing của Angular, ta sẽ sử dụng component `<app-root>` như một cái vỏ để tải nội dung các component khác vào tương ứng với mỗi mẫu URL.

- Bước 1: mở file `src/app/app.routes.ts`, thêm lệnh import sau để khai báo sử dụng RouterModule:

```
import { provideRouter } from '@angular/router';
import { routes } from './app.routes';
```

- Bước 2: Định nghĩa các đối tượng Route trong website, mỗi đối tượng Route được mô tả gồm một mẫu URL và một component. Ví dụ nếu ta có một Route có mẫu URL là “list” và component là “TaskListComponent” thì địa chỉ trang web ứng với việc hiển thị TaskListComponent là <http://localhost:4200/list>.

Như vậy, ta cần định nghĩa 3 Route cho mỗi component hiển thị các trang danh sách, thêm mới, chi tiết, cần thêm 1 Route thể hiện trang báo lỗi và một Route nữa để chuyển hướng trang gốc <http://localhost:4200> đến trang mặc định <http://localhost:4200/list>. Các định nghĩa này được viết dưới dạng các JavaScript Object và đặt trong một Array trong file `app.routes.ts`:

```
export const routes: Routes = [
  { path: 'list', component: TaskListComponent },
  { path: 'detail/:id', component: TaskDetailComponent },
  { path: 'new', component: TaskNewComponent },
  { path: '', redirectTo: '/list', pathMatch: 'full' },
  { path: '**', component: PageNotFoundComponent }
];
```

- Bước 3: truyền mảng các Route trên cho ApplicationConfig trong `app.config.ts`:

```
export const appConfig: ApplicationConfig = {
```

```
providers: [provideRouter(routes)
```

];- Bước 4: mở file /src/app/app.component.html, xóa nội dung của file này và thêm vào khai báo `<router-outlet></router-outlet>`

Trong ts của AppComponent, thêm khai báo trong metadata:

```
@Component({ selector: 'app-root', standalone: true, imports: [CommonModule, RouterOutlet, RouterLink, RouterLinkActive],  
templateUrl: './app.component.html', styleUrls: ['./app.component.css'] })
```

Kết quả ta sẽ có nội dung các file sau khi sửa như sau

app.config.ts:

```
import { ApplicationConfig, importProvidersFrom } from '@angular/core';  
import { provideRouter } from '@angular/router';
```

```
import { routes } from './app.routes';
```

```
export const appConfig: ApplicationConfig = {  
  providers: [provideRouter(routes)]  
};
```

app.routes.ts:

```
import { Routes } from '@angular/router';  
import { TaskListComponent } from  
'./task-list/task-list.component';  
import { TaskDetailComponent } from  
'./task-detail/task-detail.component';  
import { TaskNewComponent } from './task-new/task-new.component';  
import { PageNotFoundComponent } from  
'./page-not-found/page-not-found.component';
```

```
export const routes: Routes = [  
  { path: 'list', component: TaskListComponent },  
  { path: 'detail/:id', component: TaskDetailComponent },  
  { path: 'new', component: TaskNewComponent },  
  { path: '', redirectTo: '/list', pathMatch: 'full' },  
  { path: '**', component: PageNotFoundComponent }  
];
```

App.component.html

```
<router-outlet></router-outlet>
```

App.Component.ts:

```
@Component({
  selector: 'app-root',
  standalone: true,

  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css'],
  imports: [CommonModule, RouterOutlet, RouterLink,
RouterLinkActive, ChildComponent,ReactiveFormsModule]
})
export class AppComponent {
  ...
}
```

Sau bước này ta có thể thử truy cập vào mỗi trang theo các URL sau:

- Trang danh sách : <http://localhost:4200/list> hoặc <http://localhost:4200>
- Trang thêm mới: <http://localhost:4200/new>
- Trang chi tiết: <http://localhost:4200/detail/1>
- Trang báo lỗi: bất kỳ trang nào có cấu trúc URL không giống như các trang kia, VD: <http://localhost:4200/abc>

5. Xử lý các component

5.1 Tạo nguồn dữ liệu cho website:

Nguồn dữ liệu cho một website Angular thường được lấy từ một hệ thống webservice, trong bài thực hành này ta sẽ tạo một **nguồn dữ liệu giả** để phục vụ việc phát triển các component. Việc kết nối đến một nguồn dữ liệu thật sẽ được hướng dẫn ở bài thực hành sau.

Trước tiên, cần một lớp thể hiện một đối tượng “công việc”. Trong thư mục `/src/app`, tạo một file TypeScript mới có tên là **task.ts**, thiết lập nội dung cho file đó như sau:

```
export class task {
  id: number | undefined;
  title: string | undefined;
  content:string | undefined;
  status:number | undefined;
}
```

(trong Visual Studio Code chọn File/New File, thêm nội dung trên vào file, sau đó SaveAs với tên Task.ts và kiểu file là TypeScript vào thư mục /src/app của project)

Tiếp theo tạo lớp chứa danh sách công việc. Vẫn trong thư mục /src/app, tạo một file TypeScript mới tên là **mock-tasks.ts** có nội dung sau:

```
import { task } from './task';

export const TASKS: task[] = [
  { id: 1, title: 'công việc 1', content: 'nội dung công việc 1', status: 2},
  { id: 2, title: 'công việc 2', content: 'nội dung công việc 2', status: 1},
  { id: 3, title: 'công việc 3', content: 'nội dung công việc 3', status: 0},
  { id: 4, title: 'công việc 4', content: 'nội dung công việc 4', status: 2},
  { id: 5, title: 'công việc 5', content: 'nội dung công việc 5', status: 1},
  { id: 6, title: 'công việc 6', content: 'nội dung công việc 6', status: 0},
];
```

Ở các file TypeScript khác khi cần sử dụng nguồn dữ liệu này cần import lớp task và TASKS.

5.2 Xử lý trang hiển thị danh sách:

Trang hiển thị danh sách công việc được phụ trách bởi component task-list, mở các file bên trong thư mục /src/app/task-list trên Visual Studio Code: task-list.component.ts, task-list.component.html.

Các công việc cần xử lý ở trang này:

- Hiển thị danh sách công việc theo nguồn dữ liệu
- Xử lý sự kiện click vào tiêu đề mỗi công việc
- Xử lý sự kiện click vào các nút ở cột Hành động: Kết thúc, Xóa, Thực hiện
- Xử lý sự kiện click vào nút Thêm công việc mới

5.2.1 Hiển thị danh sách công việc

Ở **task-list.component.ts**, ta cần thực hiện các việc sau:

- import nguồn dữ liệu:

```
import { TASKS } from '../mock-tasks';
```

- Khai báo một thuộc tính của lớp TaskListComponent và gán nguồn dữ liệu cho thuộc tính này:

```
tasks=TASKS;
```

Ở **task-list.component.html**, ta xóa nội dung cũ và thêm đoạn mã sau:

```
<h1>Danh sách công việc</h1>

<div>

  <button>Thêm công việc mới</button>

</div>

<div>

  <table border="1">

    <tr>

      <td>STT</td>

      <td>Tên công việc</td>

      <td>Trạng thái</td>

      <td>Hành động</td>

    </tr>

    <tr *ngFor="let task of tasks">

      <td>{{task.id}}</td>

      <td>{{task.title}}</td>

      <td>

        <span *ngIf="task.status==0">Chưa làm</span>

        <span *ngIf="task.status==1">Đang làm</span>

        <span *ngIf="task.status==2">Hoàn thành</span>

      </td>

      <td>

        <button>

          <span *ngIf="task.status==0">Thực hiện</span>

          <span *ngIf="task.status==1">Kết thúc</span>

          <span *ngIf="task.status==2">Xóa</span>

        </button>

      </td>

    </tr>

  </table>

</div>
```

Giải thích:

- Danh sách công việc hiển thị ở dạng bảng nên ta sử dụng thẻ `<table>` để hiển thị, dòng đầu tiên là các tiêu đề của các cột.

- Để hiển thị mỗi công việc một dòng, ta sử dụng directive `*ngFor` để tạo vòng lặp sinh ra các thẻ `<tr>`, mỗi thẻ là 1 dòng hiển thị nội dung một công việc

+ Nội dung của Stt, tên công việc và thời hạn được hiển thị trực tiếp bằng phương pháp **interpolation** `{{ tên thuộc tính }}`

+ Nội dung của cột Trạng thái được hiển thị gián tiếp từ trường status của công việc: sử dụng directive `*ngIf` để hiển thị các nội dung tương ứng với mỗi giá trị của status. Nội dung của cột Hành động cũng được xử lý tương tự.

5.2.2 Xử lý các sự kiện

- Xử lý sự kiện click vào nút thêm công việc: trong trang `task-list.component.html`, ta cập nhật lại nút Thêm mới công việc để nó trở đến trang Thêm mới công việc khi được click:

```
<button [routerLink]="['/new']">Thêm công việc mới</button>
```

- Xử lý sự kiện click vào tiêu đề mỗi công việc: cập nhật lại phần hiển thị danh sách công việc như sau để gắn link đến trang chi tiết ở mỗi tiêu đề:

```
<tr *ngFor="let task of tasks">
  <td>{{task.id}}</td>
  <td><a [routerLink]="['/detail', task.id]">{{task.title}}</a></td>
  <td>{{task.deadline}}</td>
  <td>
    <span *ngIf="task.status==0">Chưa làm</span>
    <span *ngIf="task.status==1">Đang làm</span>
    <span *ngIf="task.status==2">Hoàn thành</span>
  </td>
  <td>
    <button>
      <span *ngIf="task.status==0">Thực hiện</span>
      <span *ngIf="task.status==1">Kết thúc</span>
      <span *ngIf="task.status==2">Xóa</span>
    </button>
  </td>
</tr>
```

- Xử lý sự kiện khi nhấn các nút Hành động: bài thực hành sau

5.2.3 Kết quả:

task-list.component.html

```
<h1>Danh sách công việc</h1>

<div>

<button [routerLink]="['/new']">Thêm công việc mới</button>

</div>

<div>

<table border="1">

<tr>

<td>STT</td>

<td>Tên công việc</td>

<td>Trạng thái</td>

<td>Hành động</td>

</tr>

<tr *ngFor="let task of tasks">

<td>{{task.id}}</td>

<td><a [routerLink]="['/detail', task.id]">{{task.title}}</a></td>

<td>

<span *ngIf="task.status==0">Chưa làm</span>

<span *ngIf="task.status==1">Đang làm</span>

<span *ngIf="task.status==2">Hoàn thành</span>

</td>

<td>

<button>

<span *ngIf="task.status==0">Thực hiện</span>

<span *ngIf="task.status==1">Kết thúc</span>

<span *ngIf="task.status==2">Xóa</span>

</button>

</td>

</tr>

</table>

</div>
```

task-list.component.ts

```
import { Component, OnInit } from '@angular/core';
import { TASKS } from '../mock-tasks';

@Component({
  selector: 'app-task-list',
```

```
templateUrl: './task-list.component.html',
styleUrls: ['./task-list.component.css']
})
export class TaskListComponent implements OnInit {
  tasks=TASKS;
  constructor() { }

  ngOnInit() {
  }
}
```

5.3 Xử lý trang thêm mới công việc:

Trang thêm mới công việc được phụ trách bởi component task-new, mở các file sau trong thư mục /src/app/task-new để làm việc: task-new.component.html, task-new.component.ts. Mở thêm file app.module.ts trong thư mục /src/app.

Các công việc cần làm:

- Tạo form nhập dữ liệu cho công việc mới
- Xử lý sự kiện Submit form
- Xử lý sự kiện nhấn nút Hủy

5.3.1 Tạo form nhập dữ liệu:

Ta sẽ sử dụng Reactive Form để tạo lập và quản lý dữ liệu trên form.

Bước 1: đăng ký module ReactiveFormsModule

Ở file app.module.ts, bổ sung import ReactiveFormsModule :

```
import { ReactiveFormsModule } from '@angular/forms';
```

Bổ sung thêm khai báo ở phần import của metadata @NgModule:

```
imports: [
  BrowserModule,
  RouterModule.forRoot(
    appRoutes,
    { enableTracing: true }
  ),
  ReactiveFormsModule
```

]

Bước 2: trong file task-new.component.ts, khai báo sử dụng FormBuilder và khởi tạo FormBuilder:

```
import { FormBuilder } from '@angular/forms';

....

export class TaskNewComponent implements OnInit {
  newForm = this.fb.group({
    title: [''],
    content: ['']
  });
  constructor(private fb: FormBuilder) { }

  ngOnInit() {
  }
}
```

- Bước 3: trên file task-new.component.html, xóa các nội dung mặc định và tạo form như sau:

```
<h1>Thêm mới công việc</h1>
<form [formGroup]="newForm">
  <div>
    <div>Tiêu đề</div>
    <div><input type="text" formControlName="title" ></div>
  </div>
  <div>
    <div>Nội dung</div>
    <div><textarea formControlName="content" ></textarea></div>
  </div>
  <div>
    <div><button >Hủy</button><button>Lưu</button></div>
  </div>
</form>
```

Giải thích: để liên kết thuộc tính của newForm trong file .ts với các thẻ <input> trên file template, ta cần gán thuộc tính formGroup của <form> với tên thuộc tính newForm và khai báo các formControlName tương ứng ở mỗi thẻ <input>.

5.3.2 Xử lý sự kiện:

- Sự kiện click nút hủy, ta thêm thiết lập routerLink là trang /list:

```
<button [routerLink]="['/list']" >Hủy</button>
```

- Sự kiện Submit của form, ta thêm hàm sau vào file task-new.component.ts:

```
onSubmit() {  
    console.log(this.newForm.value);  
}
```

Đồng thời, trên file task-new.component.html, thiết lập sự kiện Submit cho form và kiểu của nút Lưu đổi thành kiểu "submit":

```
<form [formGroup]="newForm" (ngSubmit)="onSubmit()">
```

...

```
<button type="submit">Lưu</button>
```

Nội dung xử lý thực sự của sự kiện onSubmit() sẽ được hướng dẫn ở bài thực hành sau.

5.3.3 Kết quả:

/src/app/app.module.ts

```
import { BrowserModule } from '@angular/platform-browser';  
import { NgModule } from '@angular/core';  
  
import { AppComponent } from './app.component';  
import { TaskListComponent } from './task-list/task-list.component';  
import { TaskDetailComponent } from './task-detail/task-detail.component';  
import { TaskNewComponent } from './task-new/task-new.component';  
import { PageNotFoundComponent } from './page-not-found/page-not-found.component';  
import { RouterModule, Routes } from '@angular/router';  
import { ReactiveFormsModule } from '@angular/forms';  
const appRoutes: Routes = [  
  { path: 'list', component: TaskListComponent },  
  { path: 'detail/:id', component: TaskDetailComponent },  
  { path: 'new', component: TaskNewComponent },
```

```
{ path: '', redirectTo: '/list', pathMatch: 'full' },
{ path: '**', component: PageNotFoundComponent }
];
@NgModule({
  declarations: [
    AppComponent,
    TaskListComponent,
    TaskDetailComponent,
    TaskNewComponent,
    PageNotFoundComponent
  ],
  imports: [
    BrowserModule,
    RouterModule.forRoot(
      appRoutes,
      { enableTracing: true }
    ),
    ReactiveFormsModule
  ],
  providers: [],
  bootstrap: [AppComponent]
})
export class AppModule { }
```

/src/app/task-new/task-new.component.ts

```
import { Component, OnInit } from '@angular/core';
import { FormBuilder } from '@angular/forms';
@Component({
  selector: 'app-task-new',
  templateUrl: './task-new.component.html',
  styleUrls: ['./task-new.component.css']
})
export class TaskNewComponent implements OnInit {
  newForm = this.fb.group({
    title: [''],
    content: [''],
```

```
});  
constructor(private fb: FormBuilder) { }  
ngOnInit() {  
}  
onSubmit() {  
  console.log(this.newForm.value);  
}  
}
```

/src/app/task-new/task-new.component.html

```
<h1>Thêm mới công việc</h1>  
<form [formGroup]="newForm" (ngSubmit)="onSubmit()">  
  <div>  
    <div>Tiêu đề</div>  
    <div><input type="text" formControlName="title" ></div>  
  </div>  
  <div>  
    <div>Nội dung</div>  
    <div><textarea formControlName="content" ></div>  
  </div>  
  <div>  
    <div>  
      <div><button [routerLink]="['/list']" >Hủy</button><button  
type="submit">Lưu</button></div>  
    </div>  
  </div>  
</form>
```

5.4 Xử lý trang hiển thị chi tiết công việc

Trang chi tiết được phụ trách bởi component task-detail, mở các file sau trong thư mục /src/app/task-detail: task-detail.component.html, task-detail.component.ts trong Visual Studio Code để làm việc.

Các việc phải làm:

- Hiển thị chi tiết công việc từ id nhận được khi chuyển hướng từ trang danh sách sang.
- Xử lý sự kiện cho các nút Back, Xóa và các nút hành động: Thực hiện, Kết thúc

5.4.1 Hiện thị chi tiết công việc:

Để hiển thị chi tiết công việc cần biết công việc nào được chọn khi người dùng click ở trang danh sách. Như vậy cần truyền một tham số nào đó từ trang danh sách sang ứng với mỗi công việc được chọn. Ta đã thực hiện điều đó bằng các thiết lập sau:

- Tạo Route cho trang chi tiết có URL là “detail/:id” , với “id” ở đây chính là tên của tham số được gửi kèm URL (xem lại cấu trúc trong app.module.ts).
- Gán id của công việc được chọn vào thuộc tính RouterLink của mỗi tiêu đề công việc ở trang danh sách:

```
<a [routerLink]="['/detail', task.id]">{{task.title}}</a>
```

Công việc bây giờ là cần lấy ra id của công việc được truyền qua URL, sau đó lấy ra công việc theo id nhận được rồi hiển thị các thông tin khác lên giao diện.

- Bước 1: mở file task-detail.component.ts, lấy tham số id được truyền qua snapshot của đối tượng `route` , sau đó gán cho một thuộc tính của lớp, chú ý khởi tạo đối tượng route ở hàm khởi tạo và thêm các thư viện cần thiết:

```
import { Component, OnInit } from '@angular/core';
import { ActivatedRoute } from '@angular/router';

...

export class TaskDetailComponent implements OnInit {

  id:any;
  constructor( private route: ActivatedRoute) { }

  ngOnInit() {

    this.id = this.route.snapshot.paramMap.get('id');

  }

}
```

- Bước 2, lấy thông tin của đối tượng công việc qua id và gán cho một thuộc tính kiểu `task`, lưu ý rằng ta cần phải import nguồn dữ liệu giả TASKS thì mới có danh sách đối tượng task để tìm kiếm:

```
...

import { TASKS } from '../mock-tasks';

...

export class TaskDetailComponent implements OnInit {

  id:any;
  t:any;
```

```
..  
ngOnInit() {  
    this.id = this.route.snapshot.paramMap.get('id');  
    this.t=this.findById(this.id);  
}  
findById(id:any):any{  
    return TASKS.find(obj => obj.id == id);  
}  
}
```

Hàm findById() có nhiệm vụ tìm trong mảng TASKS phần tử nào có thuộc tính id bằng giá trị truyền vào (đây là cú pháp trên JavaScript phiên bản ES6 và TypeScript)

- Bước 3: hiển thị thông tin lên template của trang chi tiết, mở file task-detail.component.html và thay bằng nội dung sau:

```
<h1>Chi tiết công việc</h1>  
<div><button>Back</button></div>  
<div>{{t.title}}</div>  
<div>Nội dung</div>  
<p>{{t.content}}</p>  
<div>  
<button>Xóa</button>  
<button *ngIf="t.status!=2">  
<span *ngIf="t.status==0">Thực hiện</span>  
<span *ngIf="t.status==1">Kết thúc</span>  
</button>  
</div>
```

Giải thích: trên template này ta vẫn sử dụng phương pháp interpolation để hiển thị các nội dung của thuộc tính **t** lên các vị trí cần hiển thị trên khuôn html. Việc hiển thị các nút hành động như Thực hiện hay Kết thúc được thực hiện bằng cách dùng directive *ngIf kiểm tra điều kiện hiển thị: chỉ hiển thị nút hành động khi trạng thái công việc là 0 (chưa làm) hoặc 1 (đang làm), không hiển thị nếu trạng thái là 2 (đã hoàn thành).

5.4.2 Xử lý sự kiện:

- Sự kiện nhấn nút Back: quay về trang danh sách, đơn giản chỉ cần thêm thuộc tính [routerLink] cho button:

```
<button [routerLink]="['/list']" >Back</button>
```

- Sự kiện nhấn nút Xóa, các nút hành động: làm trong bài thực hành sau.

5.4.3 Tổng kết:

/src/app/task-detail/task-detail.component.ts

```
import { Component, OnInit } from '@angular/core';
import { ActivatedRoute } from '@angular/router';
import { TASKS } from '../mock-task';
import { task } from '../Task';

@Component({
  selector: 'app-task-detail',
  templateUrl: './task-detail.component.html',
  styleUrls: ['./task-detail.component.css']
})
export class TaskDetailComponent implements OnInit {
  id="0";
  t;
  constructor( private route: ActivatedRoute
  ) { }

  ngOnInit() {
    this.id = this.route.snapshot.paramMap.get('id');
    this.t=this.findByID(this.id);
  }
  findByID(id):task{
    return TASKS.find(obj => obj.id == id);
  }
}

/src/app/task-detail/task-detail.component.html

<h1>Chi tiết công việc</h1>
<div><button [routerLink]="['/list']" >Back</button></div>
```

```
<div>{{t.title}}</div>
<div>Nội dung</div>
<p>{{t.content}}</p>
<div>
  <button>Xóa</button>
  <button *ngIf="t.status!=2">
    <span *ngIf="t.status==0">Thực hiện</span>
    <span *ngIf="t.status==1">Kết thúc</span>
  </button>
</div>
```

6. Style (phần mở rộng):

Sinh viên tự thiết kế style theo cách của mình.

Chú ý các file quan trọng sau khi muốn layout và style:

- /src/index.html: trang html gốc của ứng dụng web, nếu ta muốn tạo layout chung cho cả ứng dụng web thì cần làm trên file html này.
- /src/style.css: file CSS chung cho cả ứng dụng web, các luật CSS ở file này sẽ ảnh hưởng đến tất cả các component của ứng dụng.
- các file .css của mỗi component: chứa các luật CSS cho từng component.